

UNIVERSIDAD PANAMERICANA

Facultad de Ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación

Sistemas Operativos II



Sistema de Mensajería Cifrada

Eliseo Velásquez Rodas (000132837)

Quetzaltenango, junio 2026

Sistema de Mensajería Cifrada

Eliseo Velásquez Rodas (000132837)

Quetzaltenango, junio 2026

Contenido

Resumen	5
Marco Conceptual	7
1.2 Criptografía Simétrica y Asimétrica	7
1.3 Protocolo TLS y Seguridad en el Transporte	8
1.4 Hashing de Contraseñas y Almacenamiento Seguro de Credenciales	8
1.5 Arquitectura de Sistemas Distribuidos	8
1.6 Modelo de Amenazas STRIDE	8
1.7 Principios de Diseño de Software Seguro	9
Marco Metodológico	9
2.1 Enfoque de Investigación	9
2.2 Diseño Orientado a la Seguridad	9
2.3 Criterios de Evaluación de Seguridad	11
2.4 Herramientas y Tecnologías Utilizadas	11
Arquitectura del Sistema	11
3.1 Visión General de la Arquitectura	11
3.2 Organización Modular	15
3.3 Protocolo de Aplicación	15
3.4 Gestión de Concurrencia	16
Arquitectura de Seguridad	16
4.1 Modelo de Amenazas	16
4.2 Capas de Cifrado: Defensa en Profundidad	17
4.3 Gestión del Ciclo de Vida de Claves RSA	17
Persistencia y Gestión de la Concurrencia	18
5.1 Base de Datos SQLite	18
5.2 Migración Automática de Esquema	20
5.3 Modelo de Concurrencia y Mecanismos de Sincronización	21
Despliegue Distribuido	21
6.1 Entorno de Virtualización	21
6.2 Alternativa con Contenedores	22
Incidencias y Soluciones	22
Compilación, Pruebas y Ejecución	23
Conclusiones	24
Recomendaciones	25

Referencias.....	27
-------------------------	-----------

Resumen

El presente trabajo académico analiza el diseño, la arquitectura y la implementación de un Sistema de Mensajería Cifrada Distribuida desarrollado en Java, cuyo objetivo central consiste en garantizar la privacidad de las comunicaciones mediante cifrado de extremo a extremo (E2E). El sistema integra tres capas de seguridad independientes: TLS 1.2/1.3 para la protección del transporte de datos, RSA-2048 combinado con AES-256-GCM para el cifrado E2E, y BCrypt con factor de costo 12 para el almacenamiento seguro de credenciales. La arquitectura adoptada sigue el paradigma cliente-servidor centralizado, con gestión de concurrencia mediante ExecutorService y persistencia en SQLite con mensajes almacenados como blobs cifrados. La investigación aplica el modelo de amenazas STRIDE para identificar y mitigar riesgos, y fundamenta sus decisiones de diseño en estándares reconocidos, incluyendo el RFC 8446 para TLS 1.3, las recomendaciones NIST SP 800-175B y los lineamientos del OWASP ASVS 4.0. Los resultados demuestran que un sistema modular basado en principios SOLID, con múltiples capas de cifrado independientes, logra un nivel alto de confidencialidad e integridad en las comunicaciones digitales, incluso ante un servidor comprometido.

Palabras clave: cifrado de extremo a extremo, TLS, RSA-2048, AES-256-GCM, BCrypt, sistemas distribuidos, seguridad informática, Java, STRIDE, NIST.

Introducción

En el panorama contemporáneo de las comunicaciones digitales, la privacidad y la integridad de los datos transmitidos constituyen una preocupación transversal a toda organización e individuo que interactúa en redes de computadoras. La proliferación de amenazas como la interceptación de tráfico, los ataques de tipo Man-in-the-Middle (MITM) y el acceso no autorizado a bases de datos evidencia la necesidad de implementar soluciones criptográficas robustas que protejan el contenido de los mensajes desde su origen hasta su destino.

Conklin et al. (2022) señalan que los ataques MITM y la captura de tráfico no cifrado son vectores de amenaza prevalentes tanto en redes corporativas como en entornos públicos, con consecuencias que van desde la divulgación de información confidencial hasta el compromiso total de sistemas críticos. Ante esta realidad, el cifrado de extremo a extremo (E2E) emerge como el estándar más elevado de privacidad, dado que garantiza que ningún intermediario, incluido el propio servidor de mensajería, pueda acceder al contenido de las comunicaciones (Ferguson et al., 2010).

El presente trabajo analiza la arquitectura completa de un sistema de mensajería cifrada desarrollado íntegramente en Java, que integra múltiples capas de seguridad basadas en estándares internacionales. La relevancia de este análisis radica tanto en el aspecto técnico, por la integración coherente de mecanismos criptográficos heterogéneos, como en el académico, al ilustrar la aplicación práctica de principios de diseño de software seguro en un contexto real y funcional.

El trabajo se estructura en los siguientes componentes: un marco conceptual que establece las bases teóricas de la criptografía y la seguridad en sistemas distribuidos; un marco metodológico que describe el enfoque de diseño y análisis de amenazas; la descripción detallada de la arquitectura y los protocolos del sistema; y, finalmente, conclusiones y recomendaciones derivadas del proceso de investigación e implementación.

Marco Conceptual

1.1 Sistemas de Mensajería Segura

Los sistemas de mensajería segura son aplicaciones diseñadas para transmitir información entre dos o más partes garantizando, como mínimo, tres propiedades fundamentales de la seguridad informática: confidencialidad, integridad y autenticación (Stallings, 2022). La confidencialidad asegura que solo los destinatarios legítimos puedan leer el contenido; la integridad garantiza que los mensajes no sean alterados en tránsito; y la autenticación verifica la identidad de los interlocutores antes de permitir el intercambio de información.

Marlinspike y Perrin (2016) establecen que el paradigma contemporáneo de mensajería segura, ejemplificado por el protocolo Double Ratchet implementado en Signal, combina el intercambio de claves Diffie-Hellman con cadenas de claves de sesión para lograr forward secrecy y break-in recovery simultáneos. Este enfoque demuestra que la seguridad de los mensajes no depende de un único mecanismo criptográfico, sino de la correcta integración de múltiples primitivas criptográficas complementarias.

1.2 Criptografía Simétrica y Asimétrica

La criptografía de clave simétrica utiliza la misma clave para los procesos de cifrado y descifrado. Su principal ventaja es la eficiencia computacional, lo que la hace adecuada para cifrar grandes volúmenes de datos. El estándar AES (Advanced Encryption Standard) en modo GCM (Galois/Counter Mode), avalado por el NIST SP 800-38D (Dworkin, 2007), proporciona simultáneamente confidencialidad mediante un cifrador de flujo y autenticación mediante su Authentication Tag de 128 bits, lo que lo convierte en el modo de operación preferido en sistemas de alta seguridad.

La criptografía asimétrica, por su parte, utiliza un par de claves matemáticamente relacionadas: una clave pública, distribuida libremente, y una clave privada, mantenida en secreto por su propietario. El algoritmo RSA, cuya seguridad se fundamenta en la dificultad computacional de factorizar números enteros grandes, es el estándar de facto para el intercambio seguro de claves en sistemas de mensajería (Kaufman et al., 2002). El esquema híbrido, que combina RSA para el intercambio de la clave de sesión y AES para el cifrado del contenido, ofrece la fortaleza de ambos paradigmas con un costo computacional razonable.

1.3 Protocolo TLS y Seguridad en el Transporte

El protocolo TLS (Transport Layer Security), en su versión 1.3 estandarizada por el RFC 8446 (Rescorla, 2018), representa la evolución más segura de los protocolos de comunicación cifrada en redes TCP/IP. A diferencia de versiones anteriores, TLS 1.3 elimina conjuntos de cifrado obsoletos, reduce el handshake a un único round-trip y exige forward secrecy en todas las conexiones, protegiendo retroactivamente el tráfico capturado ante un eventual compromiso de claves a largo plazo.

1.4 Hashing de Contraseñas y Almacenamiento Seguro de Credenciales

El almacenamiento seguro de contraseñas requiere el uso de funciones de hash diseñadas específicamente para ser computacionalmente costosas, a diferencia de los algoritmos de hash criptográficos de propósito general como SHA-256. Provos y Mazières (1999) propusieron BCrypt como un esquema de hash adaptativo basado en la función de cifrado de bloques Blowfish, cuyo factor de costo configurable permite ajustar la dificultad del cómputo conforme avanza la capacidad del hardware disponible. Con factor de costo 12, la generación de un hash requiere aproximadamente 250-300 milisegundos, lo que hace prohibitivos los ataques de fuerza bruta masivos.

1.5 Arquitectura de Sistemas Distribuidos

Tanenbaum y Van Steen (2017) definen un sistema distribuido como una colección de computadoras independientes que se presentan ante los usuarios como un sistema único y coherente. En el contexto de la mensajería, la arquitectura cliente-servidor centralizada, aunque implica un punto único de confianza, ofrece ventajas significativas en términos de simplicidad de gestión, control de acceso y auditoría. El cifrado E2E elimina la principal desventaja de esta arquitectura al garantizar que el servidor actúe como mero enrutador de datos cifrados que no puede descifrar.

1.6 Modelo de Amenazas STRIDE

El modelo STRIDE, desarrollado originalmente por Howard y LeBlanc (2002) en Microsoft, proporciona una taxonomía estructurada para la identificación sistemática de amenazas en sistemas de software. El acrónimo comprende seis categorías: Spoofing (suplantación de identidad), Tampering (alteración de datos), Repudiation (repudio de acciones), Information Disclosure (divulgación de información), Denial of Service (denegación de servicio) y Elevation of Privilege (elevación de privilegios). Su aplicación durante el diseño del sistema permite

identificar contramedidas específicas para cada categoría de amenaza antes de que estas se materialicen.

1.7 Principios de Diseño de Software Seguro

Martin (2017) establece que una arquitectura limpia, basada en los principios SOLID y en la separación estricta de responsabilidades, no solo facilita la mantenibilidad del software, sino que también reduce la superficie de ataque al minimizar el acoplamiento entre componentes. En el contexto de la seguridad, Gamma et al. (1994) demuestran que los patrones de diseño bien establecidos, como Handler y Singleton, proporcionan soluciones probadas para problemas recurrentes de concurrencia y gestión de estado en sistemas de red. El principio de defensa en profundidad, reconocido por el NIST (2020) como una de las mejores prácticas fundamentales en seguridad de sistemas distribuidos, establece que múltiples capas de control independientes deben proteger los activos críticos, de modo que el compromiso de una capa no implique la vulneración total del sistema.

Marco Metodológico

2.1 Enfoque de Investigación

El presente trabajo emplea un enfoque mixto que combina el análisis documental de estándares internacionales de seguridad y criptografía con el estudio de caso de un sistema de mensajería cifrada implementado en Java. El análisis documental permite fundamentar las decisiones de diseño en evidencia empírica y normativa reconocida, mientras que el estudio de caso facilita la observación de la integración práctica de los conceptos teóricos en un sistema funcional.

2.2 Diseño Orientado a la Seguridad

La metodología de diseño aplicada al sistema sigue el enfoque Security by Design, en el cual las consideraciones de seguridad se integran desde las etapas más tempranas del proceso de desarrollo, en lugar de incorporarse como medidas correctivas posteriores. Este enfoque se operacionaliza mediante tres etapas principales:

2.2.1 Etapa 1 Análisis de amenazas: Se aplicó el modelo STRIDE para identificar sistemáticamente los vectores de ataque relevantes para un sistema de mensajería en red, documentando las mitigaciones correspondientes para cada categoría de amenaza identificada.

2.2.2 Etapa 2 Diseño de la arquitectura de seguridad: Basándose en los resultados del análisis de amenazas y en los estándares NIST, OWASP y RFC pertinentes, se diseñó una

arquitectura de defensa en profundidad con tres capas de cifrado independientes: TLS para el transporte, cifrado híbrido RSA/AES para el E2E, y BCrypt para las credenciales.

2.2.3 Etapa 3 Implementación y validación: El sistema fue implementado en Java con módulos Gradle independientes, siguiendo los principios SOLID de diseño orientado a objetos. La validación incluyó pruebas de concurrencia con múltiples clientes simultáneos y verificación de la integridad del cifrado en escenarios de interceptación simulada. La arquitectura reduce el acoplamiento y facilita la evolución independiente de los componentes del sistema.

Módulo	Responsabilidad Principal	Dependencias Clave
common	Protocolo JSON (Paquete/TipoPaquete), BCrypt, configuración TLS	jBCrypt, javax.net.ssl
server	Aceptación concurrente, autenticación, enrutado, SQLite, logs	common, SQLite JDBC, java.util.concurrent
client	GUI Swing, cliente consola Docker, conexión en tiempo real, claves RSA	common, javax.swing, java.security

El sistema implementa un modelo de seguridad en profundidad (Defense in Depth) con tres capas independientes de protección. Este enfoque garantiza que el compromiso de una capa no derive automáticamente en el compromiso total del sistema.

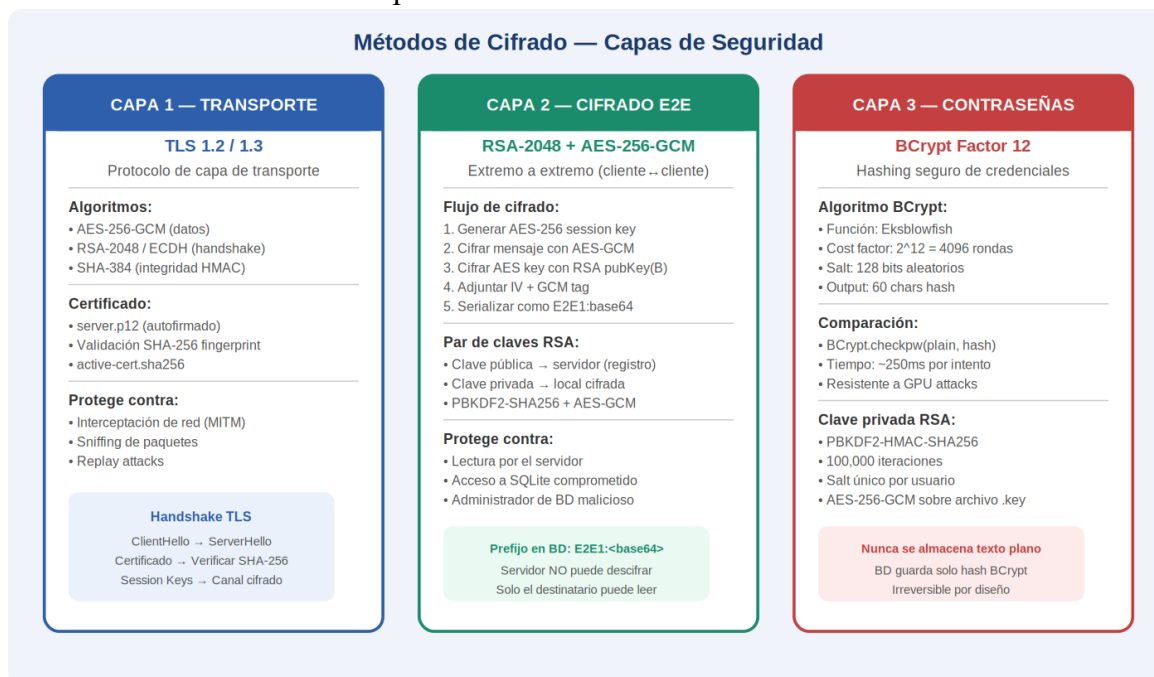


Figura 1. Diagrama de Métodos de Cifrado — Tres Capas de Seguridad

2.3 Criterios de Evaluación de Seguridad

La evaluación del nivel de seguridad alcanzado por el sistema se realizó conforme a tres marcos de referencia reconocidos internacionalmente. En primer lugar, el OWASP Application Security Verification Standard (ASVS) 4.0 fue utilizado para verificar los requisitos de autenticación, gestión de sesiones y criptografía. En segundo lugar, las guías del NIST para selección de algoritmos criptográficos, incluyendo las publicaciones SP 800-175B y SP 800-132, sirvieron como criterio de conformidad para los algoritmos implementados. En tercer lugar, los principios del modelo de Zero Trust Architecture (NIST, 2020) orientaron el diseño de la validación de identidad, bajo el supuesto de que ningún componente del sistema es inherentemente confiable.

2.4 Herramientas y Tecnologías Utilizadas

La implementación del sistema se realizó utilizando Java 11 LTS como plataforma base, Gradle 8.7 como sistema de construcción y gestión de dependencias, SQLite con el driver JDBC como motor de persistencia, la biblioteca jBCrypt 0.4 para el hashing de contraseñas, y la API estándar de Java Security (javax.net.ssl, java.security) para las operaciones criptográficas. La organización del código en tres módulos independientes (common, server, client) refleja la aplicación del principio de separación de responsabilidades (Martin, 2017).

Arquitectura del Sistema

3.1 Visión General de la Arquitectura

El sistema adopta un patrón de arquitectura cliente-servidor centralizada con comunicación cifrada mediante TCP sobre TLS. Tanenbaum y Van Steen (2017) señalan que las arquitecturas cliente-servidor siguen siendo predominantes en sistemas de mensajería por su facilidad de gestión y control de acceso, siendo el cifrado E2E el mecanismo que compensa su principal desventaja en materia de privacidad. La comunicación entre cliente y servidor se realiza mediante mensajes JSON serializados en formato JSON Lines, lo que facilita el parsing secuencial y la extensibilidad del protocolo.

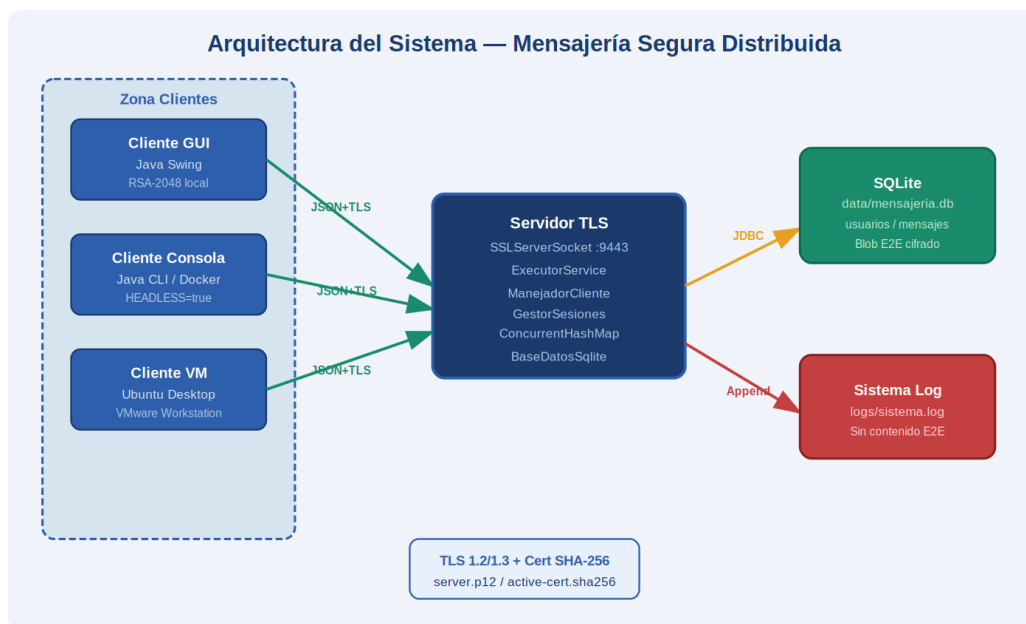


Figura 2. Diagrama de Arquitectura General del Sistema

3.1.1 Diagrama de Clases:

El modelo de clases refleja una clara separación entre los módulos funcionales, siguiendo principios SOLID. La clase ManejadorCliente implementa la interfaz Runnable para su ejecución en el pool de hilos, mientras que GestorSesiones mantiene el estado de autenticación de forma thread-safe mediante ConcurrentHashMap.

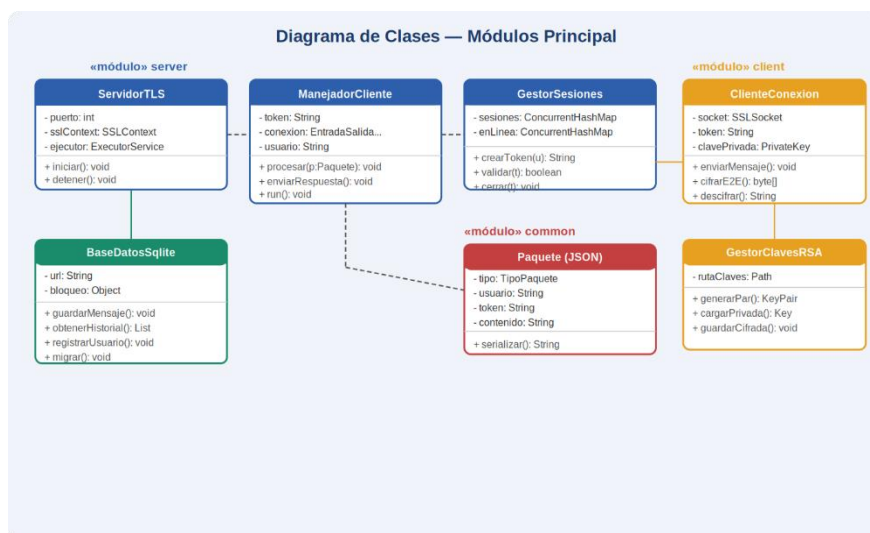


Figura 3. Diagrama de Clases — Módulos server, common y client

3.1.2 Diagrama de Secuencia:

El diagrama de secuencia ilustra el flujo completo de un mensaje cifrado entre dos clientes, incluyendo las validaciones del servidor y el push en tiempo real. Es importante destacar que el servidor actúa únicamente como enrutador del blob cifrado, sin capacidad de descifrar el contenido.

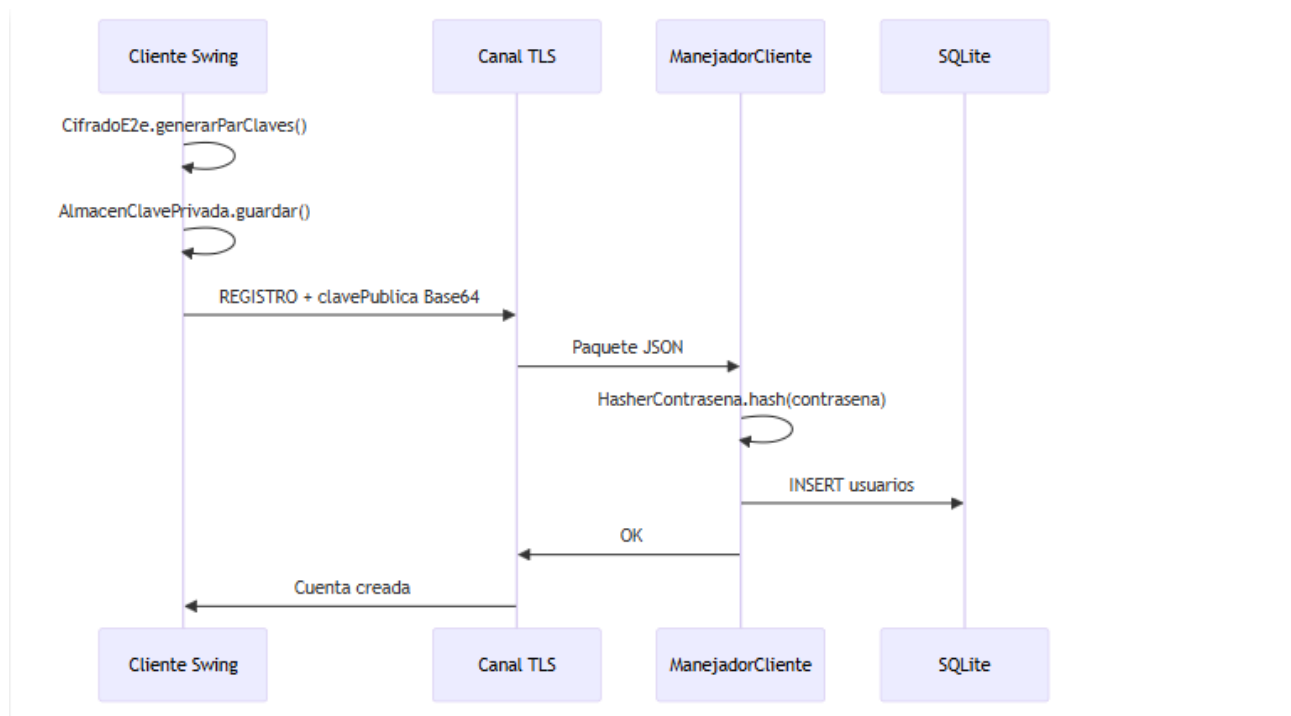


Figura 4.1. Diagrama de Secuencia — Registro

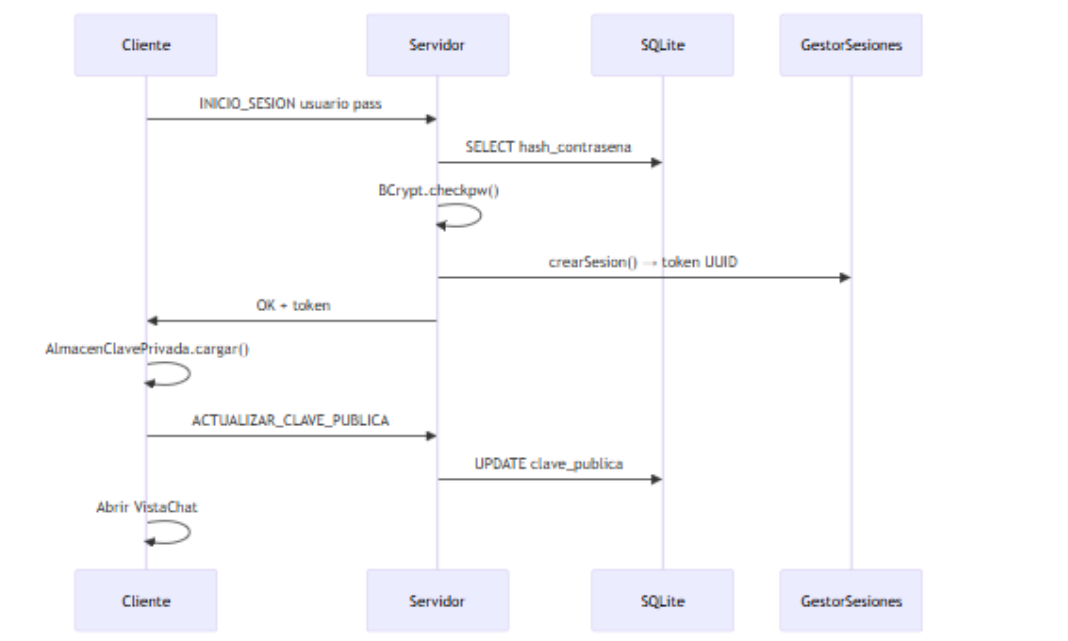


Figura 4.2. Diagrama de Secuencia — Inicio de Sesión

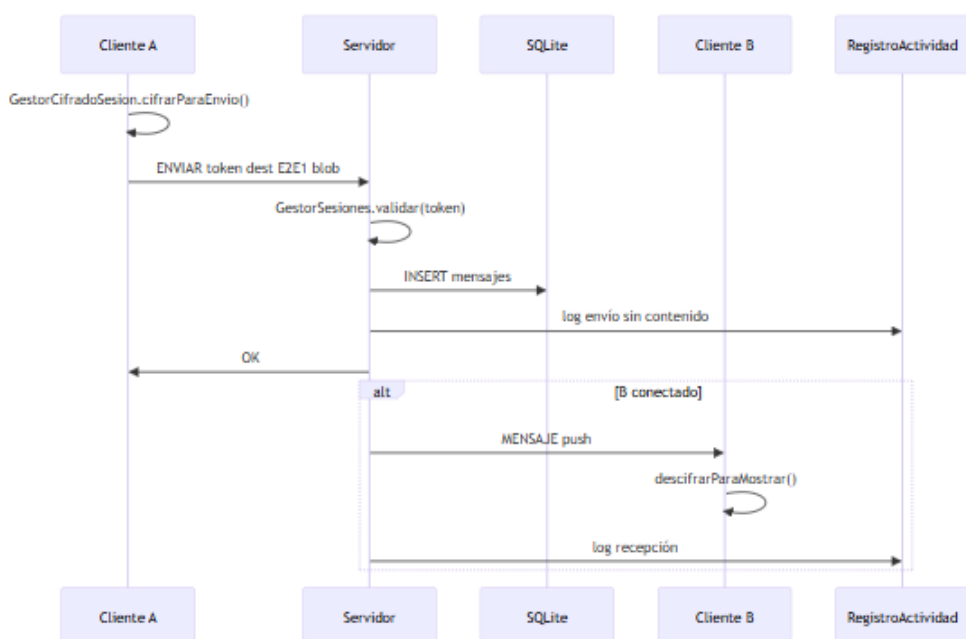


Figura 4.3. Diagrama de Secuencia — Envío de Mensaje con Cifrado E2E

3.1.2 Diagrama de Casos de Uso:

Los casos de uso identifican los actores principales del sistema el Usuario y el Administrador y sus interacciones con las funcionalidades disponibles. El caso de uso UC03 (Enviar Mensaje E2E) incluye una relación «include» con UC02 (Iniciar Sesión), dado que la autenticación es un prerequisite para cualquier operación de envío.

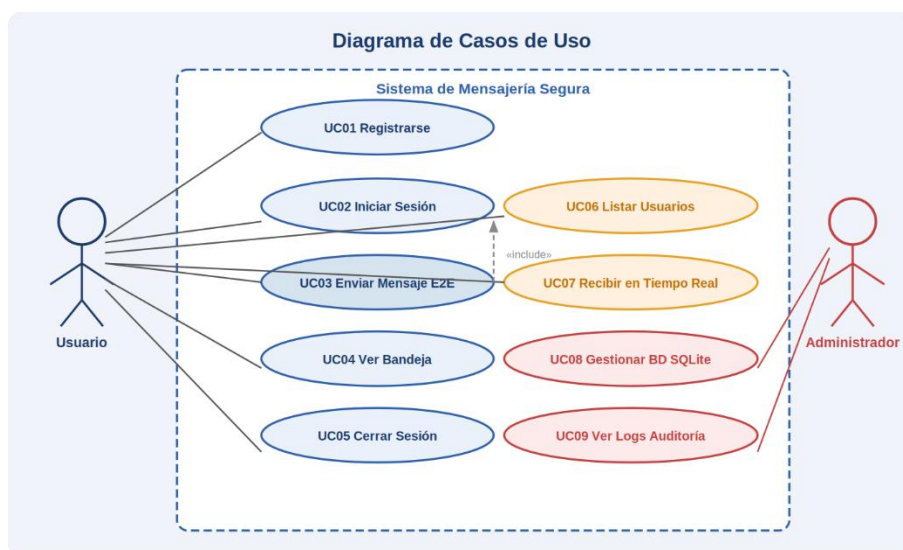


Figura 5. Diagrama de Casos de Uso del Sistema

3.2 Organización Modular

Martin (2017) establece que la arquitectura modular, con fronteras bien definidas entre capas, reduce el acoplamiento y facilita la evolución independiente de los componentes. En consonancia con este principio, el proyecto se organiza en tres módulos Gradle independientes:

3.2.1 Módulo common: Contiene las definiciones del protocolo de comunicación (Paquete, TipoPaquete), la lógica de cifrado BCrypt y la configuración TLS compartida entre cliente y servidor.

3.2.2 Módulo server: Implementa la aceptación concurrente de conexiones TLS, la autenticación de usuarios, el enrutado de mensajes cifrados y la persistencia en SQLite.

3.2.3 Módulo client: Proporciona la interfaz gráfica en Swing, la generación y gestión del par de claves RSA, y la lógica de cifrado/descifrado E2E en el extremo del cliente.

3.3 Protocolo de Aplicación

El protocolo de aplicación sigue los principios de los protocolos de texto plano estructurado, comparables en concepción a HTTP y SMTP (Fielding & Reschke, 2022). Cada

unidad de protocolo, denominada Paquete, contiene un campo discriminador tipo que determina el procesamiento correspondiente en el servidor. Los tipos de paquete incluyen operaciones de registro e inicio de sesión, sin autenticación previa, y operaciones que requieren un token UUID de sesión, como el envío de mensajes, la consulta de bandeja de entrada y la gestión de usuarios.

3.4 Gestión de Concurrencia

Goetz et al. (2006) establecen que el modelo de un hilo por cliente, gestionado mediante `ExecutorService`, es el patrón canónico para servidores de red en Java que atienden múltiples conexiones simultáneas. El sistema implementa este modelo mediante un pool de hilos dinámico (`Executors.newCachedThreadPool()`), que reutiliza hilos disponibles y crea nuevos solo cuando la demanda lo requiere. La consistencia del estado compartido se garantiza mediante `ConcurrentHashMap` para el registro de sesiones activas y bloques `synchronized` para el acceso serializado a SQLite y la escritura atómica en sockets.

Arquitectura de Seguridad

4.1 Modelo de Amenazas

Howard y LeBlanc (2002) propusieron el modelo STRIDE como un marco sistemático para identificar amenazas de seguridad en sistemas de software durante las etapas de diseño. La aplicación de este modelo al sistema de mensajería permitió identificar seis categorías de amenaza y sus correspondientes mitigaciones:

La amenaza de Spoofing, o suplantación de identidad, es mitigada mediante la combinación de hashing BCrypt para la verificación de credenciales y la asignación de tokens UUID únicos por sesión, que deben ser presentados en cada operación que requiera autenticación. La amenaza de Tampering, o alteración de datos en tránsito, es contrarrestada por TLS para el canal de transporte y por el Authentication Tag de 128 bits del modo GCM para la integridad de cada mensaje cifrado E2E.

La amenaza de Information Disclosure, especialmente relevante en un sistema de mensajería, es mitigada por el cifrado E2E basado en RSA-2048 y AES-256-GCM, que garantiza que el servidor, incluso si es comprometido, solo accede a blobs cifrados sin la capacidad de descifrarlos. La Denegación de Servicio es contenida por el pool de hilos dinámico, que limita el impacto de conexiones masivas. Finalmente, la Elevación de Privilegios es prevenida mediante la validación del token de sesión en cada operación que modifica el estado del sistema.

4.2 Capas de Cifrado: Defensa en Profundidad

El NIST (2020) reconoce la defensa en profundidad como una de las mejores prácticas fundamentales en seguridad de sistemas distribuidos, estableciendo que múltiples capas de control independientes deben proteger los activos críticos. El sistema implementa tres capas de cifrado ortogonales que garantizan que el compromiso de cualquiera de ellas no implique la vulneración total de la confidencialidad.

4.2.1 Capa 1 — TLS 1.3 (Transporte): Rescorla (2018) documenta que TLS 1.3 elimina los conjuntos de cifrado con RSA key exchange sin forward secrecy, presentes en versiones anteriores, y reduce la latencia del handshake. El sistema utiliza un certificado autofirmado con validación explícita por huella SHA-256, lo que impide los ataques de sustitución de certificado sin necesidad de una infraestructura PKI completa.

4.2.2 Capa 2 — Cifrado Híbrido E2E (RSA-2048 + AES-256-GCM): Stallings (2022) establece que el esquema de cifrado híbrido es el estándar en sistemas de mensajería seguros, combinando la seguridad del intercambio de claves asimétrico con la eficiencia del cifrado simétrico. El modo GCM, recomendado por Dworkin (2007) en el NIST SP 800-38D, proporciona autenticación de datos mediante su Authentication Tag sin requerir una operación criptográfica adicional.

4.2.3 Capa 3 — BCrypt para Credenciales: Provos y Mazières (1999) diseñaron BCrypt para ser resistente a hardware especializado como FPGAs y GPUs, manteniendo su relevancia mediante el ajuste del factor de costo. Con factor 12, el sistema logra un equilibrio entre la usabilidad del sistema, con tiempos de respuesta de 250-300 ms por operación de autenticación, y la resistencia a ataques de diccionario masivos.

4.3 Gestión del Ciclo de Vida de Claves RSA

Kaufman et al. (2002) señalan que la correcta gestión del ciclo de vida de las claves criptográficas es tan importante como la fortaleza de los algoritmos utilizados. El sistema implementa un ciclo de vida completo para las claves RSA: durante el registro, se genera un par RSA-2048 mediante `java.security.KeyPairGenerator`, la clave pública se envía al servidor para almacenamiento en la base de datos, y la clave privada se cifra localmente mediante PBKDF2-SHA256 combinado con AES-256-GCM antes de su persistencia en disco. Durante el envío de mensajes, la clave pública del destinatario es recuperada del servidor para cifrar la clave de sesión AES, y el mensaje completo se almacena como un blob E2E1 en SQLite. El descifrado ocurre

exclusivamente en la memoria del cliente destinatario, sin que el contenido en claro esté jamás disponible en el servidor.

La gestión de claves es uno de los aspectos más críticos en cualquier sistema criptográfico. El sistema implementa un ciclo de vida completo para las claves RSA: generación, distribución de la clave pública, almacenamiento seguro de la clave privada y uso en operaciones de cifrado/descifrado.

Fase	Operación	Mecanismo	Almacenamiento
Registro	Generación del par RSA-2048	java.security.KeyPairGenerator	Par en memoria temporal
Registro	Envío de clave pública	Campo clavePublica en REGISTRO	Servidor (tabla usuarios)
Registro	Cifrado de clave privada	PBKDF2-SHA256 + AES-256-GCM	data/claves/{usuario}.key
Envío	Cifrado del mensaje	RSA-OAEP-SHA256 sobre AES key	Blob E2E1: en SQLite
Recepción	Descifrado del mensaje	RSA privKey + AES-GCM decrypt	Solo en memoria cliente

Persistencia y Gestión de la Concurrencia

5.1 Base de Datos SQLite

La elección de SQLite como motor de persistencia responde a la necesidad de una solución embebida, sin dependencias externas adicionales, que provea transacciones ACID y soporte SQL estándar. Owens y Allen (2010) documentan que SQLite es uno de los motores de base de datos más ampliamente desplegados del mundo, con aplicaciones que van desde dispositivos embebidos hasta sistemas de mensajería a gran escala, lo que avala su confiabilidad en contextos de producción real.

El esquema de base de datos comprende dos tablas principales: usuarios, que almacena identificadores, credenciales hasheadas, claves públicas RSA y metadatos de registro; y mensajes, que almacena los blobs cifrados E2E con el prefijo E2E1, junto con los metadatos de remitente, destinatario, timestamp y estado de lectura. Notablemente, el contenido de los mensajes nunca se almacena en texto claro, lo que garantiza que un compromiso de la base de datos no implique la exposición del contenido de las comunicaciones.

La elección de SQLite como motor de base de datos responde a la necesidad de una solución embebida, sin dependencias externas adicionales, que provea transacciones ACID y soporte SQL estándar. Para aplicaciones de mensajería con carga moderada, SQLite ofrece un rendimiento adecuado, siendo utilizado en producción por sistemas como WhatsApp para el almacenamiento local de mensajes.

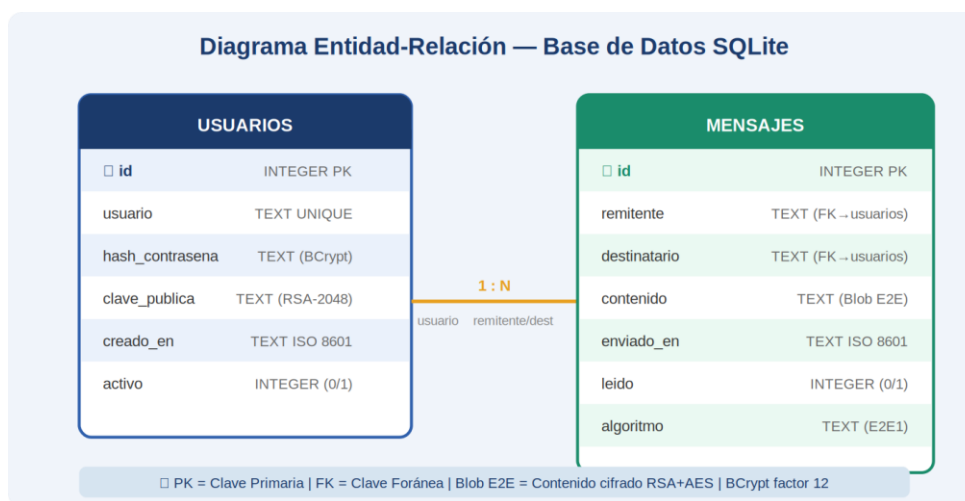


Figura 6. Diagrama Entidad-Relación — Base de Datos SQLite

5.1.1 Diccionario de Datos

Tabla Usuarios

Campo	Tipo SQLite	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único autoincremental
usuario	TEXT	NOT NULL, UNIQUE	Nombre de usuario en minúsculas
hash_contraseña	TEXT	NOT NULL	Hash BCrypt-12 (60 caracteres)
clave_publica	TEXT	NOT NULL	Clave pública RSA-2048 en Base64
creado_en	TEXT	NOT NULL	Timestamp ISO 8601 de registro
activo	INTEGER	DEFAULT 1	Flag de cuenta activa (0/1)

Tabla Mensajes

Campo	Tipo SQLite	Restricciones	Descripción
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Identificador único autoincremental
remitente	TEXT	NOT NULL, FK→usuarios	Nombre del emisor
destinatario	TEXT	NOT NULL, FK→usuarios	Nombre del destinatario
contenido	TEXT	NOT NULL	Blob cifrado E2E con prefijo E2E1:

Campo	Tipo SQLite	Restricciones	Descripción
enviado_en	TEXT	NOT NULL	Timestamp ISO 8601 de envío
leído	INTEGER	DEFAULT 0	Flag de lectura (0=no leído, 1=leído)

5.1.2 Diagrama de Flujo (Autenticación)

El diagrama de flujo siguiente ilustra el proceso completo de conexión y autenticación de un cliente, desde el establecimiento del canal TLS hasta la obtención del token de sesión que habilita el resto de operaciones.

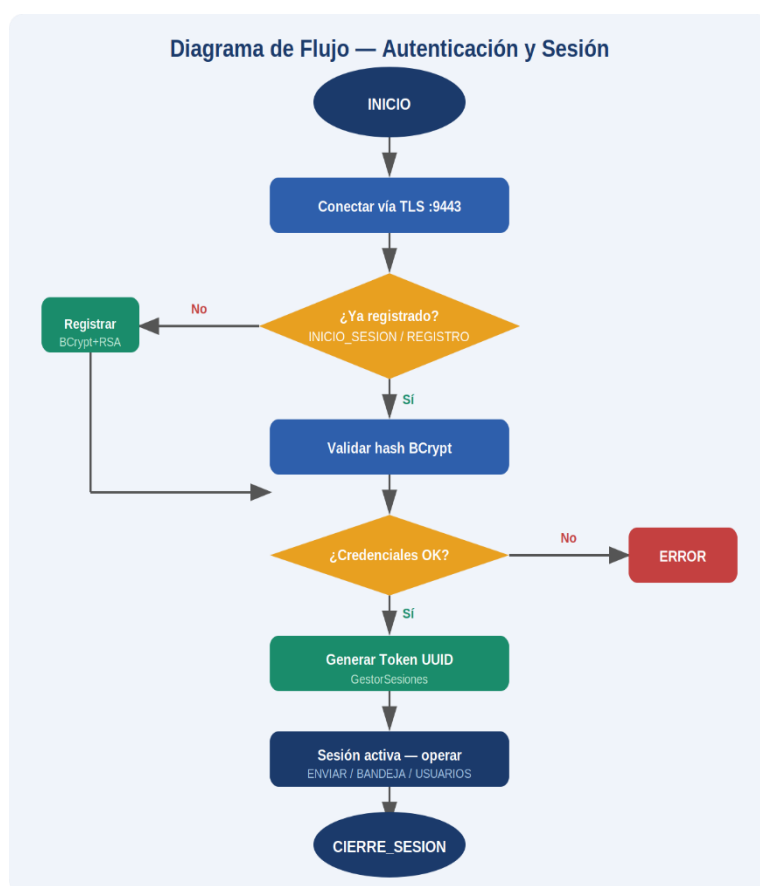


Figura 7. Diagrama de Flujo — Proceso de Autenticación y Gestión de Sesión

5.2 Migración Automática de Esquema

Redgate (2023) establece que las migraciones de base de datos automáticas y versionadas son una práctica esencial en el ciclo de vida del software, permitiendo evolucionar el esquema sin pérdida de datos y con trazabilidad completa de los cambios. El sistema implementa una migración

forward-only que detecta la presencia de tablas en inglés de versiones previas, replica los datos en el nuevo esquema en español y elimina las tablas obsoletas, garantizando la compatibilidad con instalaciones existentes.

5.3 Modelo de Concurrencia y Mecanismos de Sincronización

La gestión de concurrencia en el servidor implementa un modelo de un hilo por cliente, delegado a un `ExecutorService` con pool de hilos dinámico. Este enfoque, combinado con estructuras de datos thread-safe (`ConcurrentHashMap`), garantiza la consistencia del estado compartido sin introducir puntos de contención excesivos.

Silberschatz et al. (2018) señalan que los mecanismos de sincronización son fundamentales para garantizar la consistencia en accesos concurrentes a recursos compartidos. El sistema implementa una estrategia de sincronización diferenciada según el recurso: `ConcurrentHashMap` sin bloqueos explícitos para los mapas de sesiones y usuarios activos, cuyos accesos concurrentes son seguros por diseño; bloques `synchronized` sobre un objeto de bloqueo dedicado para las operaciones en SQLite, que serializa los accesos y previene condiciones de carrera; y métodos `synchronized` en la clase de escritura al socket, que garantiza la atomicidad de cada mensaje transmitido y evita el intercalado de paquetes de distintos hilos.

Mecanismo	Clase Java	Propósito	Garantías
<code>SSLServerSocket.accept()</code>	<code>ServidorTLS</code>	Aceptación de conexiones entrantes TLS	Thread-safe, bloqueante
<code>ExecutorService</code> (pool dinámico)	<code>Executors.newCachedThreadPool()</code>	Pool de hilos para <code>ManejadorCliente</code>	Reutilización y escalado automático
<code>ConcurrentHashMap</code> (sesiones)	<code>GestorSesiones</code>	Mapa de tokens UUID activos	Lectura/escritura concurrente sin locks
<code>ConcurrentHashMap</code> (online)	<code>GestorSesiones</code>	Usuarios conectados en tiempo real	Visibilidad inmediata entre hilos
Bloque <code>synchronized</code> (SQLite)	<code>BaseDatosSqlite.bloqueo</code>	Serialización de acceso a SQLite	Prevención de condiciones de carrera
<code>synchronized</code> (socket write)	<code>EntradaSalidaConexion</code>	Escritura atómica al socket TCP	Sin intercalado de mensajes

Despliegue Distribuido

6.1 Entorno de Virtualización

El método de despliegue principal utiliza VMware Workstation con máquinas virtuales Ubuntu Desktop, lo cual proporciona un entorno de red aislado y controlado que simula un

despliegue en red corporativa real. La virtualización permite replicar topologías de red complejas en hardware único de desarrollo.

El método de despliegue principal utiliza VMware Workstation con máquinas virtuales Ubuntu Desktop, proporcionando un entorno de red aislado que simula un despliegue corporativo real. VMware (2023) documenta que la virtualización de red permite replicar topologías complejas en hardware único de desarrollo, lo que facilita la validación de sistemas distribuidos antes del despliegue en infraestructura de producción.

Rol	Sistema	Proceso	Puerto/Protocolo
Servidor TLS	VM Ubuntu Desktop	./iniciar-servidor.sh → Monitor Swing	9443/TCP (TLS)
Cliente 1..N	VM(s) Ubuntu Desktop	./iniciar-cliente.sh → GUI Swing	Efímero → 9443
Validación TLS	Todos los nodos	certs/active-cert.sha256 (SHA-256)	Verificación en handshake
Administración BD	VM Servidor	ver-admin-base-datos.bat / Gradle	Local (SQLite)

6.2 Alternativa con Contenedores

Burns et al. (2019) establecen que la contenerización mediante Docker permite la reproducibilidad del entorno de ejecución, eliminando problemas de dependencias de sistema operativo y facilitando la automatización de pruebas en pipelines de integración continua. Para entornos sin interfaz gráfica, el sistema ofrece soporte para Docker Compose con clientes en modo consola, activado mediante la variable HEADLESS=true. Humble y Farley (2010) señalan que la documentación sistemática de incidencias técnicas y sus soluciones reduce el tiempo de resolución de problemas recurrentes, práctica reflejada en la tabla de incidencias conocidas documentada durante el desarrollo del sistema.

Incidencias y Soluciones

El proceso de desarrollo identificó diversas incidencias técnicas cuya resolución generó soluciones que se documentan a continuación como parte del conocimiento técnico del proyecto.

Incidencia	Causa Raíz	Impacto	Solución Implementada
Certificado no confiable	Certificado autofirmado rechazado por defecto	Conexión TLS bloqueada	Validación explícita por huella SHA-256 en active-cert.sha256

Incidencia	Causa Raíz	Impacto	Solución Implementada
GUI en contenedor Docker	Ausencia de servidor X11 en entornos headless	Cliente Swing no arranca	Modo HEADLESS=true con cliente consola; VMs Ubuntu para GUI
Gradle 9 incompatible	APIs deprecadas en Gradle 9 removidas	Fallo en compilación	Versión fijada a Gradle 8.7 mediante gradlew.bat
Carrera lectura/escritura socket	Múltiples hilos escribiendo en el mismo socket	Mensajes intercalados/corruptos	Escritura sincronizada en EntradaSalidaConexion
Esquema BD en inglés	Versiones previas usaban nombres en inglés	Incompatibilidad de datos históricos	Migración automática al iniciar servidor (BaseDatosSqlite)
Pérdida de sesión abrupta	Desconexión sin CIERRE_SESION explícito	Token huérfano en GestorSesiones	Captura de IOException en hilo lector para invalidar token

Compilación, Pruebas y Ejecución

Requisitos del Entorno:

Componente	Versión Mínima	Observación
Java JDK	11 LTS	OpenJDK o Oracle JDK; requerido en servidor y clientes
Gradle	8.7 (via gradlew)	Gradle 9 incompatible por APIs deprecadas
SQLite JDBC	3.x	Incluido como dependencia Gradle
jBCrypt	0.4	Biblioteca para hashing BCrypt
VMware Workstation	17.x (opcional)	Para despliegue distribuido en VM
Docker + Compose	24.x (opcional)	Para modo headless sin GUI

Conclusiones

El análisis del Sistema de Mensajería Cifrada Distribuida permite derivar las siguientes conclusiones fundamentales sobre la integración de seguridad criptográfica en sistemas de software de red:

La arquitectura de defensa en profundidad, con tres capas de cifrado independientes (TLS, E2E híbrido y BCrypt), demuestra ser un enfoque técnicamente sólido para garantizar la confidencialidad de las comunicaciones en un sistema de mensajería centralizado. El principio clave es la ortogonalidad de las capas: el compromiso de cualquiera de ellas no implica automáticamente la exposición del contenido de los mensajes, dado que las capas restantes mantienen su protección de forma independiente.

El modelo de amenazas STRIDE, aplicado durante las etapas tempranas del diseño, permite identificar sistemáticamente los vectores de ataque relevantes y diseñar contramedidas específicas para cada categoría. Este enfoque proactivo, en contraste con la incorporación de medidas de seguridad posteriores al diseño, resulta en un sistema cuyas decisiones arquitectónicas están fundamentadas en un análisis de riesgo explícito y documentado.

La selección de estándares criptográficos reconocidos internacionalmente, específicamente TLS 1.3 (RFC 8446), AES-256-GCM (NIST SP 800-38D), RSA-2048 con OAEP-SHA256 y BCrypt con factor de costo 12, garantiza que la seguridad del sistema no depende de algoritmos propietarios o diseños criptográficos no auditados. Esta adherencia a estándares es fundamental para la auditabilidad y la confianza en sistemas de seguridad crítica.

La arquitectura modular basada en principios SOLID y la organización del código en módulos Gradle independientes no solo facilita la mantenibilidad del sistema, sino que también reduce la superficie de ataque al establecer fronteras claras entre componentes con distintos niveles de confianza. El servidor actúa exclusivamente como enrutador de datos cifrados, sin acceso a las claves privadas de los clientes, lo que limita el impacto de un potencial compromiso del servidor.

La gestión correcta del ciclo de vida de las claves criptográficas, incluyendo la generación segura, el almacenamiento cifrado de la clave privada con PBKDF2+AES-GCM y la distribución de la clave pública mediante el servidor, representa uno de los aspectos más complejos y críticos de cualquier sistema criptográfico. La experiencia demuestra que las vulnerabilidades en sistemas criptográficos rara vez residen en los algoritmos mismos, sino en su implementación e integración.

Recomendaciones

Con base en el análisis realizado y en las conclusiones derivadas, se formulan las siguientes recomendaciones para el fortalecimiento y evolución del sistema:

Implementar Perfect Forward Secrecy a nivel de aplicación. Aunque TLS 1.3 garantiza forward secrecy en el transporte, el esquema E2E actual utiliza claves RSA estáticas por usuario. Se recomienda evolucionar hacia un protocolo de intercambio de claves de sesión derivadas, similar al Double Ratchet documentado por Marlinspike y Perrin (2016), que garantice que el compromiso de la clave privada RSA no exponga retroactivamente mensajes anteriores.

Migrar a una infraestructura PKI con autoridad certificadora. La validación de certificados mediante huella SHA-256 codificada en archivo es un mecanismo funcional, pero operacionalmente frágil en entornos con múltiples servidores o renovaciones frecuentes de certificados. Se recomienda implementar una infraestructura PKI interna con una autoridad certificadora propia, lo que simplificaría la gestión de certificados y permitiría la revocación mediante listas CRL o el protocolo OCSP.

Adoptar Argon2 como función de hash para contraseñas. Aunque BCrypt con factor 12 proporciona protección adecuada en el hardware actual, se recomienda considerar la migración a Argon2id, ganador de la Password Hashing Competition en 2015, que ofrece resistencia superior a ataques con GPUs y ASICs mediante su configuración de memoria adaptable. Stallings (2022) señala que Argon2 es el algoritmo recomendado por OWASP para nuevas implementaciones de almacenamiento de contraseñas.

Implementar auditoría forense completa y detección de anomalías. El registro de auditoría actual preserva la privacidad al excluir el contenido de los mensajes, pero podría enriquecerse con metadatos de seguridad como intentos de autenticación fallidos, patrones de conexión inusuales y errores de validación TLS. La implementación de un sistema de detección de anomalías basado en estos metadatos permitiría identificar ataques en curso sin comprometer la privacidad del contenido, en línea con los principios de Zero Trust Architecture documentados por el NIST (2020).

Evolucionar hacia una arquitectura de microservicios con redundancia. La arquitectura monolítica actual presenta un punto único de fallo en el servidor central. Se recomienda evolucionar hacia una arquitectura de microservicios que separe los servicios de autenticación, enrutado de mensajes y persistencia, desplegables de forma independiente con redundancia activa.

Burns et al. (2019) documentan que Kubernetes proporciona las primitivas necesarias para orquestar este tipo de arquitecturas con alta disponibilidad y escalado automático.

Establecer un programa de pruebas de penetración periódicas. Conklin et al. (2022) establecen que la evaluación continua de la postura de seguridad, mediante pruebas de penetración realizadas por equipos independientes, es esencial para identificar vulnerabilidades que el equipo de desarrollo podría pasar por alto. Se recomienda establecer un programa de pruebas de penetración semestrales que cubra específicamente los mecanismos de autenticación, el manejo de claves criptográficas y la resistencia a ataques de denegación de servicio.

Referencias

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2.^a ed.). Addison-Wesley.
- Burns, B., Beda, J., & Hightower, K. (2019). *Kubernetes: Up and Running* (2.^a ed.). O'Reilly Media.
- Conklin, W. A., White, G., Williams, D., & Davis, R. (2022). *Principles of Computer Security: CompTIA Security+ and Beyond* (6.^a ed.). McGraw-Hill Education.
- Dworkin, M. J. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (SP 800-38D). National Institute of Standards and Technology.
- Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley Publishing.
- Fielding, R. T., & Reschke, J. (2022). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content* (RFC 7231). IETF.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
- Howard, M., & LeBlanc, D. (2002). *Writing Secure Code* (2.^a ed.). Microsoft Press.
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). *The Unified Software Development Process*. Addison-Wesley.
- Kaufman, C., Perlman, R., & Speciner, M. (2002). *Network Security: Private Communication in a Public World* (2.^a ed.). Prentice Hall.
- Marlinspike, M., & Perrin, T. (2016). The Double Ratchet Algorithm. Signal Messenger LLC. <https://signal.org/docs/specifications/doubleratchet/>
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- NIST. (2020). Zero Trust Architecture (SP 800-207). National Institute of Standards and Technology.
- Owens, M., & Allen, G. (2010). *The Definitive Guide to SQLite* (2.^a ed.). Apress.

- Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference. USENIX Association.
- Redgate. (2023). *Database DevOps: Best Practices for Database Change Management*. Redgate Software.
- Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3 (RFC 8446). IETF.
- Schneier, B. (2015). *Data and Goliath: The Hidden Battles to Collect Your Data and Control Your World*. W. W. Norton & Company.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley.
- Stallings, W. (2022). *Cryptography and Network Security: Principles and Practice* (8.^a ed.). Pearson Education.
- Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3.^a ed.). Pearson.
- VMware, Inc. (2023). VMware Workstation Pro Documentation. <https://docs.vmware.com/en/VMware-Workstation-Pro/>